applied sciences

MDPI

Article

Personal Identification Using Embedded Raspberry Pi-Based Face Recognition Systems

Sebastian Pecolt [1], Andrzej Błażejewski [1], Tomasz Królikowski [1], Igor Maciejewski [1], Kacper Gierula [1] and Sebastian Glowinski [2,*]

[1] Faculty of Mechanical Engineering and Power Engineering, Koszalin University of Technology, Sniadeckich 2, 75453 Koszalin, Poland; sebastian.pecolt@tu.koszalin.pl (S.P.); andrzej.blazejewski@tu.koszalin.pl (A.B.); tomasz.krolikowski@tu.koszalin.pl (T.K.); igor.maciejewski@tu.koszalin.pl (I.M.); kacper.gierula@tu.koszalin.pl (K.G.)

[2] Institute of Health Sciences, Slupsk Pomeranian University, Westerplatte 64, 76200 Slupsk, Poland

* Correspondence: sebastian.glowinski@tu.koszalin.pl; Tel.: +48-943478395

Featured Application: The Raspberry Pi-based facial recognition system offers practical applications in access control for homes, businesses, and industrial sites, providing an affordable and reliable security solution. Its portability makes it ideal for surveillance in resource-limited settings. Additionally, it can enhance consumer electronics by enabling personalized smart home experiences. With further development, the system could also support demographic analysis in public spaces, contributing to informed decision making.

Abstract: Facial recognition technology has significantly advanced in recent years, with promising applications in fields ranging from security to consumer electronics. Its importance extends beyond convenience, offering enhanced security measures for sensitive areas and seamless user experiences in everyday devices. This study focuses on the development and validation of a facial recognition system utilizing a Haar cascade classifier and the AdaBoost machine learning algorithm. The system leverages characteristic facial features—distinct, measurable attributes used to identify and differentiate faces within images. A biometric facial recognition system was implemented on a Raspberry Pi microcomputer, capable of detecting and identifying faces using a self-contained reference image database. Verification involved selecting the similarity threshold, a critical factor influencing the balance between accuracy, security, and user experience in biometric systems. Testing under various environmental conditions, facial expressions, and user demographics confirmed the system's accuracy and efficiency, achieving an average recognition time of 10.5 s under different lighting conditions, such as daylight, artificial light, and low-light scenarios. It is shown that the system's accuracy and scalability can be enhanced through testing with larger databases, hardware upgrades like higher-resolution cameras, and advanced deep learning algorithms to address challenges such as extreme facial angles. Threshold optimization tests with six male participants revealed a value that effectively balances accuracy and efficiency. While the system performed effectively under controlled conditions, challenges such as biometric similarities and vulnerabilities to spoofing with printed photos underscore the need for additional security measures, such as thermal imaging. Potential applications include access control, surveillance, and statistical data collection, highlighting the system's versatility and relevance.

Keywords: AdaBoost algorithm; biometric features; facial recognition; Haar classifier; machine learning; security system

check for updates

Academic Editor: Alexander N. Pisarchik

Received: 20 December 2024
Revised: 12 January 2025
Accepted: 13 January 2025
Published: 17 January 2025

Citation: Pecolt, S.; Błażejewski, A.; Królikowski, T.; Maciejewski, I.; Gierula, K.; Glowinski, S. Personal Identification Using Embedded Raspberry Pi-Based Face Recognition Systems. *Appl. Sci.* **2025**, *15*, 887. https://doi.org/10.3390/app15020887

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Facial recognition technology has experienced remarkable advancements in recent years, unlocking diverse applications across fields ranging from security to consumer electronics. The origins of facial recognition research date back to the 1960s, when early methods relied on analyzing facial proportions and features [1]. By the 1990s, more advanced techniques, such as Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA), emerged, enabling more effective matching of facial images to stored patterns in databases [2,3]. The 21st century saw the rise of sophisticated methods, including local binary pattern histograms (LBPHs) and Fisherfaces, further enhancing the accuracy and robustness of facial recognition systems [4]. The significance of facial recognition extends beyond convenience, offering secure, user-friendly solutions across various domains. Biometric systems have become integral to smartphones for secure unlocking and to banking systems for transaction authentication, eliminating the reliance on traditional methods like passwords and PINs [5]. Public security and surveillance extensively leverage this technology for monitoring public areas and airports, aiding in identifying persons of interest [6–8]. Commercial applications include customer behavior analysis in e-commerce, personalized advertising, and enhanced customer service [9–11]. In banking and finance, facial recognition systems support ATMs and remote client authentication, boosting transaction security [12–14]. Face recognition techniques have evolved through three primary approaches:

- Geometric Methods: These early techniques analyzed the geometry of facial features, such as distances between the eyes, nose, and mouth. While straightforward, they struggled with variations in lighting, angles, and facial expressions [15–17].
- Feature-Based Methods: Techniques like LBPH and Gabor filters analyze textures and structures on the face's surface, offering greater resilience to changes in appearance and diversity of facial features [18–23].
- Deep Learning Methods: With advancements in neural networks, particularly Convolutional Neural Networks (CNNs), facial recognition achieved unprecedented precision [24–30]. Models like DeepFace, FaceNet, and OpenFace excel in recognizing faces even in large datasets and challenging conditions, such as low light, varied expressions, and partial occlusions.

Despite its benefits, facial recognition technology faces critical challenges. Privacy concerns, particularly in public surveillance, raise ethical questions about its misuse by corporations and governments [8,31–35]. Algorithmic bias and discrimination are also significant issues, as studies reveal disparities in recognition accuracy based on skin color, gender, and ethnicity. Efforts by major companies to address these biases underline the importance of fairness in such systems [36–40]. Additionally, variable conditions, such as lighting, occlusions (e.g., masks), and appearance changes (e.g., makeup, glasses), continue to affect recognition accuracy [41–44]. Current research aims to address these challenges by improving accuracy, minimizing algorithmic bias, and expanding applications [45,46]. For instance, emotion recognition and mental state analysis have found utility in fields such as healthcare, human resources, and marketing [36,47–50]. The future of facial recognition systems hinges on social acceptance, ethical considerations, and legal frameworks to protect user privacy and ensure equitable use.

In this study, we focus on developing a facial recognition system that balances accuracy, efficiency, and affordability. Utilizing the Haar cascade classifier for rapid feature detection and the AdaBoost algorithm for enhanced precision, we implemented the system on a Raspberry Pi microcomputer [51–55]. Our testing encompassed diverse environmental conditions, facial expressions, and user demographics, achieving an average recognition time of 10.5 s. While the system demonstrated reliable performance, challenges such as

vulnerabilities to biometric similarities and spoofing highlight the need for additional security measures, such as integrating thermal imaging. This paper discusses the development, testing, and potential applications of the system, emphasizing its strengths and limitations.

This study is critical as it explores the application of low-cost, accessible hardware like the Raspberry Pi in facial recognition systems, addressing real-world concerns such as privacy, bias, and performance under variable conditions. Mainstream solutions often rely on expensive hardware or cloud processing, which raises privacy concerns and limits accessibility. This research contributes by demonstrating how a compact and affordable device can deliver effective results while maintaining privacy through local processing. Unlike studies that depend on high-end hardware or cloud-based solutions, this research emphasizes leveraging the Raspberry Pi—a low-power, cost-effective platform—to make facial recognition more accessible.

Many existing studies utilize cloud processing, which raises data security and privacy concerns. In contrast, this study ensures that all data are processed locally on the Raspberry Pi, reducing vulnerabilities associated with external data handling. It uniquely addresses variable conditions such as lighting changes, facial angles, and occlusions, testing the robustness of the system in scenarios that mimic real-life applications. By incorporating diverse datasets, this research explicitly tackles algorithmic bias and discrimination, an aspect often overlooked in other studies, particularly those focused on advanced hardware solutions.

The strengths of this approach are evident in several aspects. The use of the Raspberry Pi makes this system affordable and widely accessible, lowering the barrier to implementing facial recognition in various fields. Local data processing enhances privacy, making it suitable for applications in sensitive areas such as home security and personal automation. Additionally, the modularity of the Raspberry Pi allows for incremental upgrades, such as adding GPU acceleration or integrating higher-resolution cameras, enabling the system to scale with user needs. Testing under variable conditions ensures reliability in real-world scenarios, adding credibility to the system's practical utility.

Nevertheless, some weaknesses reveal themselves in computational limitations. While the Raspberry Pi 3 provides sufficient power for basic applications, it struggles with the real-time processing of large datasets or multiple faces without optimization or external hardware support. Algorithmic constraints are apparent as the system relies on basic models like Haar cascades, which are less effective for extreme facial angles or complex scenarios compared to advanced deep learning-based approaches. Although efforts were made to include diverse datasets, the relatively small scale of testing (e.g., six participants for threshold determination) may limit the generalizability of the findings. The performance under challenging conditions still requires extended research. Variable conditions such as poor lighting or severe occlusions remain challenging, necessitating further enhancement of hardware or algorithmic robustness. Future work includes exploring higher-resolution cameras and expanding the user database to improve the system's robustness and applicability in real-world scenarios.

This paper is structured as follows: Section 2 explains the methodology, focusing on the implementation of the Haar cascade classifier and the AdaBoost algorithm on a Raspberry Pi microcomputer. It covers the system architecture, the training process, and the selection of the similarity threshold. Section 3 outlines the results, detailing system performance under various conditions, including different environmental settings, facial expressions, and user demographics. Section 4 discusses the findings, highlighting the system's strengths, limitations, and practical applications. In Discussion, we propose future improvements, such as integrating thermal imaging and using higher-resolution cameras, to address current challenges and expand potential applications. Finally, in the Conclusion

we summarize the main outcomes and suggest directions for future research in embedded facial recognition systems.

2. Materials and Methods

2.1. Characteristics Facial Features

The face is the main feature of appearance that distinguishes us from each other. In most cases, this is what we remember when we meet new people, and we associate them mainly with this feature of their appearance. Each of us has probably had a situation when we met a person on the street whose face seemed familiar, but we could not remember his name and surname. It is this feature of appearance that we instinctively pay attention to first to identify a person. The human mind can recognize a face even from a distance when details are blurred. This means that the shape, size, or arrangement of organs on the face are sufficient to identify it. We can easily describe the characteristics of organs by locating characteristic points on the face image. Point selection is crucial for face recognition. We should choose those that represent the most important facial biometric features and are easy to "extract" from the photo.

There are two groups of biometric features (Figure 1):

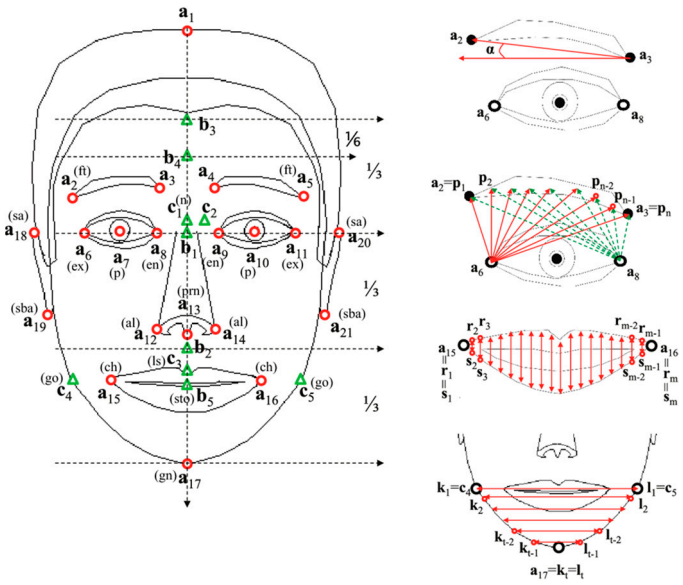


Figure 1. Examples of characteristic points representing facial biometric features [56].

(a) Anthropometric:

- Distance between the nose and eyes;
- Distance between the centers of the eyes;
- Distance between the mouth line and the eye line;
- Distance between the farthest points of the eyes;
- Distance from the center of the mouth to the furthest point of the eye.

(b) Geometric:

- Lip shape;
- Nose shape;
- Chin shape;
- Forehead shape;
- Shape of the ears;
- Face oval.

By comparing biometric features in a group of people, we are able to determine the degree of similarity between them [1]. Several factors come into play when comparing the same face in two different photos. First, it is essential to select appropriate facial landmarks. Additionally, ensuring consistent environmental conditions—such as background and lighting—is crucial. The face's spatial orientation in the photo matters too. To achieve accurate comparisons, an algorithm performs normalization procedures. These include rotations around the X, Y, and Z axes, displacement in the XY plane (Figure 2), and rescaling of the examined image. However, remember that these additional operations increase computational demands, potentially impacting system efficiency [4].

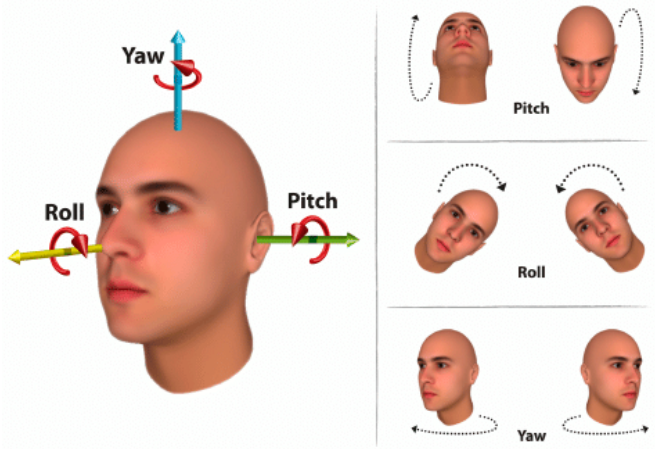


Figure 2. Axes of face rotation in an image in 3D space [57].

The critical issue is determining the appropriate tolerance within which the examined image may deviate from the norm, which involves establishing the limits within which anthropometric distances and facial contours may differ [36].

2.2. Biomimetic Facial Recognition System

The primary function of a biometric facial recognition system is to verify and identify individuals using facial photos. These systems operate in two modes: verification (Figure 3) and identification (Figure 4). Both involve comparing biometric features in the examined image with a reference image. However, the critical difference lies in how the reference image is selected. In verification mode, the user chooses the reference image. The system's sole task is to compare the examined photo with this reference photo and determine their degree of similarity. The program operates based on a comparator principle.

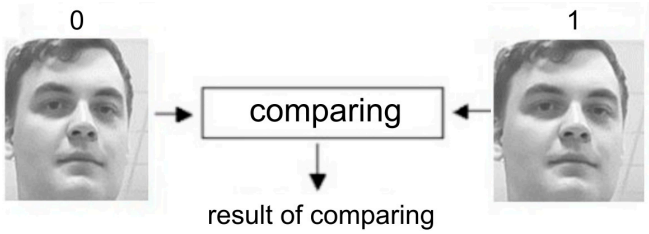


Figure 3. General diagram of how the facial recognition system works in verification mode.



Figure 4. General operation diagram of the facial recognition system in identification mode [15].

The most common task of verification mode is to authorize access. This mode is becoming increasingly popular as a system for unlocking Android mobile phones, authorizing banking and ATM transactions, and, in companies, accessing rooms with internal company information [12,13].

In the identification mode, the system compares the examined photo with a database of reference photos. Unlike the verification mode, which checks for similarity, identification provides a specific result—a particular reference photo. The identification process leverages the verification mode by comparing each database photo with the examined image. Based on a similarity measure, the program calculates the result. However, a crucial condition for obtaining a result is that the degree of similarity must exceed a specified threshold.

In identification mode, a facial recognition program performs significantly more calculations than verification mode, impacting program performance. To address this, methods are employed to reduce the initial dimensions of the feature matrix derived from reference images while retaining essential data for the identification process. This involves removing extraneous information from reference photos, such as background elements or irrelevant points that do not represent critical biometric features of the face. A specialized algorithm extracts only the points corresponding to these essential features, as described in the preceding subsection. Identification mode is commonly used to identify individuals with multiple faces in videos. To achieve this, an additional face detection algorithm is applied to the video recording, extracting all faces within the entire video frame. The algorithm should provide the program with the minor possible data matrix containing only face pixels to enhance system efficiency. This allows the subsequent algorithm to accurately determine the characteristic biometric points on those faces [36].

2.3. Face Detection

In any face recognition program, automatic face detection in input images is crucial. If not executed correctly, it can hinder further analysis. Several popular face detection algorithms include:

- Kawulok–Szymanek Algorithm: This method employs the Hough transform to detect image ellipses. The results are then verified using a support vector machine [58].
- Deformable Part Model (DPM) Algorithm: this utilizes the Histogram of Oriented Gradients (HOG) for face detection [59].
- Viola–Jones Algorithm: this approach involves feature selection and AdaBoost classification [60].
- Zhu–Ramanan Algorithm: this creates local models characterizing facial structure, leveraging biometric features regardless of head angle [4].
- Maximum Margin Object Detection Algorithm (MMOD): using a support vector machine, MMOD employs face classification based on the HOG descriptor [61].

2.4. Locating Facial Landmarks

The next stage of the program is an algorithm whose task is to identify and locate biometric features in the face area obtained using the face detection algorithm, which will enable geometric normalization. This will allow you to determine local features used to classify and create a data matrix. The most commonly used algorithms for locating facial landmarks are as follows [62]:

- An algorithm using a set of regression trees;
- CFSS algorithm—uses the assessment of the face oval, and then the final facial contour is adjusted using the regression method;
- Zhu–Ramanan algorithm—in addition to detecting faces, it also returns the coordinates of matched models.

2.5. Geometric Normalization and Classification of Facial Landmarks

The general principle of operation of facial recognition programs valid in our system is presented in Figure 5. A significant challenge in the classification of biometric features arises from discrepancies in resolution and lighting between reference images and those under examination, as well as variations in head rotation angles. These factors negatively impact classification efficiency, thereby reducing the accuracy of recognition results. Most facial recognition algorithms are optimized for frontal facial images. However, real-world applications frequently involve tilted or non-frontal head positions, necessitating geometric normalization to mitigate these issues. One geometric normalization approach involves identifying specific facial landmarks, such as the centers of the eyes, and applying image rotation and scaling operations to align these points with their counterparts in the reference image. While effective, this method is limited in its ability to accommodate facial expression variability, which can result in differing positions for features such as the mouth, nose, or eyebrows across images of the same individual.

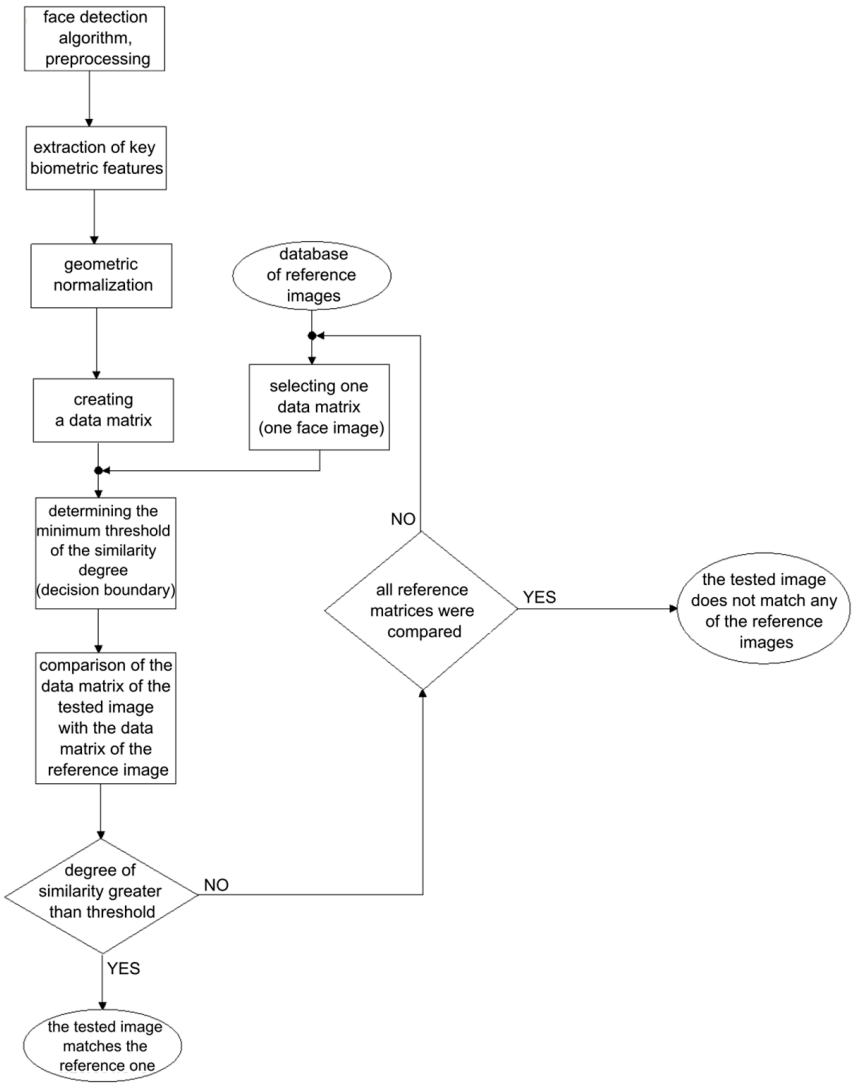


Figure 5. The general principle of operation of facial recognition programs.

Modern face recognition systems address these limitations by employing advanced geometric normalization techniques that incorporate facial expression analysis. These methods segment the detected facial region into smaller sub-areas, often centered around key landmarks, using a grid-based framework. Each sub-area undergoes independent scaling

and rotation operations to align its features with the reference image, generating distinct feature vectors for each sub-area rather than a single generalized vector. Additionally, pixel neighborhood analysis is employed to capture local brightness variations. Each pixel is compared to the brightness levels of its eight neighboring pixels, assigning binary values of 1 or 0 based on whether the central pixel's brightness is higher or lower than its neighbors. This process yields an eight-bit binary code representing the local characteristics of the pixel, forming a robust descriptor for each sub-area.

However, these advanced normalization methods increase computational complexity and memory requirements due to the need to store more extensive reference image data. They may also lead to oversized feature matrices, where the feature set of the examined face is substantially larger than that of the reference data, reducing the effectiveness of the classifier. To address this, dimensionality reduction algorithms are applied to optimize feature selection. Commonly used methods include the following:

- Partial Least Squares Regression (PLSR) [63];
- Random forest (RF) feature selection [64];
- Principal Component Analysis (PCA) [65].

The classification of facial biometric features often employs the support vector machine (SVM), recognized for its superior performance compared to algorithms such as logistic regression and decision trees. SVM is particularly effective in recognizing facial expressions and emotions.

SVM works by segmenting the dataset based on margins, defined as the distances between the closest feature points of different classes. The algorithm identifies an optimal hyperplane with maximum margins, referred to as the boundary hyperplane, which ensures robust separation of classes. One of the key strengths of SVM is its ability to transform into a non-linear classifier using the "kernel trick". This technique involves defining a kernel function to map feature vectors x and y into a higher-dimensional space, enabling the classifier to operate effectively in non-linear domains while maintaining computational efficiency [65].

2.6. Face Classification

2.6.1. AdaBoost Machine Learning

AdaBoost combines weak classifier algorithms to create a strong classifier. A single algorithm may misclassify objects. However, if multiple classifiers select the training set at each iteration and assign appropriate weight in the final decision, we can obtain accurate results from the overall classifier. It will retrain the algorithm iteratively, selecting a training set based on the accuracy of previous training [66].

Each weak classifier is trained using a random subset of the overall training set. After training, each classifier is assigned a weight based on accuracy. More accurate classifiers are assigned higher weights to have a more significant impact on the final result. The classification is based on the following formula:

$$H(x) = \operatorname{sgn}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right) \quad (1)$$

where

$h_t(x)$—the output of the t-th weak classifier for or the same example, which also takes values $+1$ or -1 depending on the predicted class.

α_t—the weight applied to the classifier determined by AdaBoost.

T—the total number of iterations or weak classifiers used to construct the final strong classifier.

$H(x)$ represents the aggregated decision of all T weak classifiers. The sign function ensures that $H(x)$ outputs $+1$ or -1, corresponding to the predicted class. This binary classification framework simplifies decision making and aligns with the fundamental principles of AdaBoost. Parameter T is a hyperparameter of the AdaBoost algorithm and is typically determined based on the specific application or dataset. The value of T directly influences the balance between underfitting and overfitting. If T is too small, the algorithm may underfit the data, resulting in lower accuracy. If T is too large, the algorithm may overfit the training data, potentially reducing generalization to unseen data. During each iteration t (where $T = 1, 2, \ldots, T$), the AdaBoost algorithm trains a new weak classifier $h_t(x)$, assigns it a weight α_t, and updates the training weights. The final classifier $H(x)$ combines the outputs of all T weak classifiers, weighted by their respective α_t, as shown in Equation (1).

Classifiers are trained individually. After training each classifier, the probability of training examples appearing in the training set for the next classifier is updated. The first classifier ($t = 1$) is trained with equal probability given for all training examples. After training, we calculate the output weight (α) for this classifier based on the following formula:

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) \tag{2}$$

where

ϵ_t—the weighted error rate of the weak classifier $h_t(x)$.

However, this formula exhibits asymptotic behavior for $\epsilon_t = 0$ or $\epsilon_t = 1$, where α_t approaches $+\infty$ or $\pm\infty$, respectively. To address this, a small constant δ is introduced to constrain ϵ_t within a valid range, such as

$$\epsilon_t = max(\delta, min(1-\delta, \epsilon_t)) \tag{3}$$

where

δ—small positive value (e.g., $\delta = 10^{-10}$).

This adjustment ensures numerical stability while maintaining the theoretical properties of the AdaBoost algorithm. For $\epsilon_t = 0$, ϵ_t is replaced by δ, avoiding an infinite value for α_t. For $\epsilon_t = 1$, ϵ_t is replaced by $1-\delta$, ensuring that α_t remains finite and does not diverge to negative infinity. This approach is standard in most practical implementations of AdaBoost and guarantees that the algorithm operates reliably even in edge cases.

The output weight is based on the classifier's error rate, which is the number of misclassifications in the training set divided by its size.

The classifier weight increases exponentially (Figure 6) as the error approaches 0. Better classifiers receive exponentially greater weight. When the weight is zero, the error rate equals 0.5. A classifier with 50% accuracy or less is not considered better than random guessing, so it is discarded. After calculating α for the first classifier, we update the weight of the training examples using the Formula (4). The argument i refers to the index of a specific training example in the dataset. Each training example is represented by a pair (x_i, y_i). The weight vector D_t is updated at each iteration t accordingly:

$$D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t} \tag{4}$$

where

$D_t(i)$—the weight vector of the i-th training example.

y_i—the true label of the i-th training sample, typically encoded +1 for the positive class and -1 for the negative class in binary classification problems.

α_t—the weight of the t-th weak classifier, determined by its accuracy (2).

Z_t—the normalization factor ensuring that the weights reach a total of 1, given by the following:

$$Z_t = \sum_{i=1}^{n} D_t(i) e^{-\alpha_t y_i h_t(x_i)} \quad (5)$$

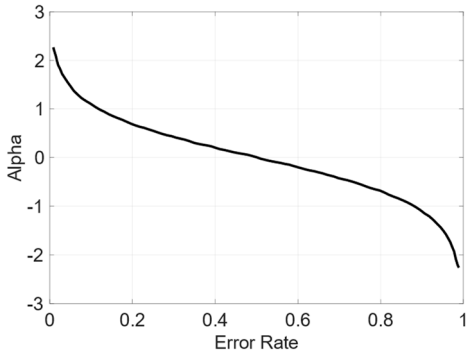


Figure 6. Graph of the dependency of the classifier weight on the error rate.

The underlying concept is to adjust the importance of each training example based on the performance of the weak classifier h_t. If $y_i h_t(x_i) > 0$ (i.e., the classifier correctly predicts y_i), the weight $D_{t+1}(i)$ decreases. If $y_i h_t(x_i) < 0$ (i.e., the classifier misclassifies y_i), the weight $D_{t+1}(i)$ increases. This mechanism ensures that subsequent weak classifiers focus more on examples that were previously misclassified, thereby improving the overall accuracy of the strong classifier.

The weights $D_t(i)$ are always non-negative and normalized, ensuring that $0 \leq D_t(i) \leq 1$ and $\sum_{i=1}^{n} D_t(i) = 1$ for all t. While individual weights $D_t(i)$ may oscillate or shift dynamically depending on the performance of the weak classifiers $h_t(x)$, the overall algorithm exhibits exponential convergence in reducing training error. Specifically, AdaBoost minimizes the training error E, as follows:

$$E = \prod_{t=1}^{T} Z_t \quad (6)$$

where Z_t is always less than 1 if $h_t(x)$ achieves accuracy better than random guessing (error $\epsilon_t < 0.5$). Thus, E decreases exponentially with increasing T. In practice, the exponential nature of the weights ensures that extreme misclassifications are penalized, but normalization (Z_t) prevents unbounded growth. The iterative process guarantees that the training error approaches zero if $\epsilon_t < 0.5$ for all t. These properties demonstrate that the weight vector D_t adapts dynamically to emphasize harder examples while maintaining boundedness and promoting convergence of the training process.

One of the most significant applications of AdaBoost is the Viola–Jones face detection algorithm. This detector uses a "cascade of rejections" consisting of multiple layers of classifiers. If, at any layer, a detection window is not recognized as a face, it is discarded, and the algorithm proceeds to the next window. The first classifier in the cascade aims to reject as many negative windows as possible at minimal computational cost. In this context, AdaBoost serves two roles. Each layer of the cascade is a strong classifier built from a combination of weaker classifiers and is used to find the most suitable features to apply in each cascade layer [67].

2.6.2. Cascading Haar Classifier

The Haar cascade classifier is a machine learning algorithm for object detection in images or video files. It is based on the concept of features proposed by Paul Viola and

Michael Jones in their 2001 paper "Rapid Object Detection using a Boosted Cascade of Simple Features". This approach involves training the cascade function with many positive and negative images, which is then used to detect objects in other images.

The algorithm consists of the following steps:

- The selection of Haar-like features;
- The creation of integral images;
- Training with the AdaBoost algorithm;
- The formation of cascade classifiers.

Initially, the algorithm requires many positive images (with faces) and negative images (without faces) to train the classifier. Next, Haar-like features are extracted. Each feature is a single value obtained by subtracting the sum of pixels under the white rectangle from the sum of pixels under the black rectangle. These features consider the neighboring areas at a specific location in the window and the intensity of pixels in each region and calculate the difference in sums.

There are several types of Haar-like features (Figure 7a), each designed to capture different patterns in the image:

- Edge Features: These features detect edges by comparing the sum of pixel intensities in two adjacent rectangular regions. For example, a vertical edge feature might compare the sum of pixels on the left side of a rectangle with the sum of pixels on the right side.
- Line Features: These features detect lines by comparing the sum of pixel intensities in three adjacent rectangular regions. For example, a horizontal line feature might compare the sum of pixels in the top, middle, and bottom regions of a rectangle.
- Four-Rectangle Features: These features detect diagonal patterns by comparing the sum of pixel intensities in four rectangular regions arranged in a grid.

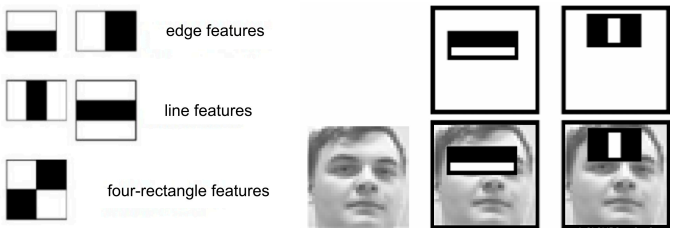


Figure 7. Haar features (**a**); positive Haar-like features on a face image example (**b**) [67].

To calculate a Haar-like feature, you subtract the sum of pixel intensities in the white regions from the sum of pixel intensities in the black regions. This can be achieved efficiently using an integral image.

The integral image I at a location (x,y) is defined as the sum of all pixel values above and to the left of (x,y):

$$I(x,y) = \sum_{x' \le x,\, y' \le y} i(x',\, y') \tag{7}$$

where $i(x', y')$ is the pixel value at (x', y').

Using the integral image, the sum of pixel values in any rectangular region can be computed in constant time, which significantly speeds up the calculation of Haar-like features. Each feature's possible sizes and locations are used to calculate many functions. For example, a 24 × 24-pixel window can generate over 160,000 features. The sums of pixels under the white and black rectangles are calculated for each feature. During the detection phase, the target image window is moved over the input image, and Haar-like features are computed for each image sub-section. However, among all the calculated features, most of them are insignificant. In Figure 7b, you can see that the top row shows

two positive features: the first selected feature seems to focus on the property that the eye region is often darker than the nose and cheeks area.

The second feature chosen finds a darker eye area than the nose bridge area. The same windows applied to the cheeks are already insignificant. This is because the top row (Figure 7b) represents biometric features of the face, while shaded areas on the cheeks are not distinguishing features between faces. AdaBoost learning algorithms are used to differentiate significant features from insignificant ones.

To this end, we apply each feature to all training images. For each feature, the best threshold is found to classify faces as positive or negative. It is impossible to avoid errors or misclassifications, so features with the minimum error rate are chosen, meaning those that classify face images most accurately. Each image initially has the same weight. After each classification, the weight of misclassified images is increased. The process is then repeated, new error rates and weights are calculated, and the process continues until the required accuracy is achieved or the required minimum number of features is found [13]. Each single Haar feature is only a "weak classifier". They are called weak because they cannot correctly classify an image. Therefore, a large number of Haar features are necessary to describe an object with enough accuracy to identify the image correctly. A cascade of weak classifiers (single Haar features) is introduced to form a single "strong" classifier.

The cascade classifier consists of a series of stages, each being a set of weak learning stimuli, i.e., simple classifiers called decision stumps. Each stage is trained using a technique called boosting. Boosting enables the training of a highly accurate classifier by taking a weighted average of the decisions made by the decision stumps. Each stage of the classifier designates the area defined by the current location of the sliding window as positive or negative. Positive indicates that an object has been found, and negative means no objects have been found. If the factor is negative, the classification of this area is completed, and the detector moves the window to the next location. The classifier passes the area to the next stage if the factor is positive. The detector then re-examines the same window at a later stage for reverification. If the window is positively verified, the final stage saves the window with a positive outcome. The stages aim to reject negative samples as quickly as possible. For the classifier to function correctly, it must have a low rate of false negatives—samples incorrectly classified as negative. If a stage incorrectly marks an object as negative, classification is halted, and the error cannot be corrected. However, each stage can have many false positives—negative samples are incorrectly classified as positive. Subsequent stages correct the errors of false positive samples, so the greater the number of stages, the fewer windows incorrectly classified as positive. The final classifier is thus a weighted sum of weak classifiers. According to the creators of the classifier, even 200 functions can ensure face detection with 95% accuracy (Figure 8). By using AdaBoost learning algorithms, we can reduce and select as significant about 6000 functions out of over 160,000 in a 24 × 24-pixel window without reducing the accuracy of the classifier [14].

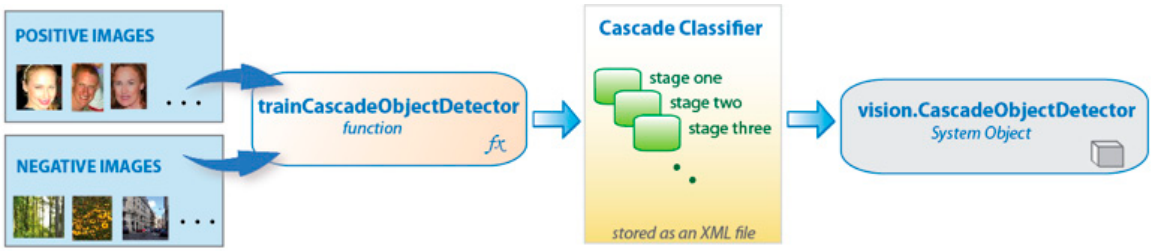
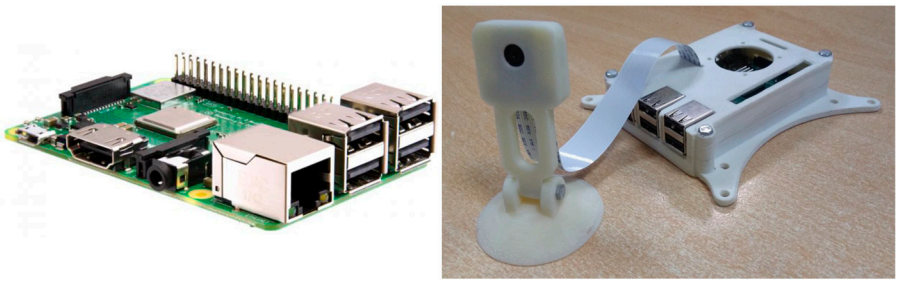


Figure 8. Haar classifier training scheme [68].

2.7. Hardware Setup

The face recognition application was written in Python on the Raspbian system, running on the Raspberry Pi 3 B+ model (Figure 9a) (Raspberry Pi Foundation, Cambridge, UK). A protective case (Figure 9b) was designed for this type of computer, allowing it to be mounted on any flat surface. It has ventilation holes for cooling the processor and covers all ports to protect them. Due to the application's requirements and the use of a dedicated camera module for Raspberry, i.e., the Raspberry Pi Rev v2.1 camera (Raspberry Pi Foundation, Cambridge, UK) a special cover (Figure 9a) was designed for the camera along with a base, allowing us to conveniently set the camera on a flat surface in a vertical direction. The camera has an 8-megapixel sensor with a native resolution of 3280 × 2464 pixels. The camera module's board housing is attached to the base with a joint, allowing free angle manipulation. All parts were printed using the FFD method on a 3D printer from ABS material.



(**a**) (**b**)

Figure 9. Raspberry Pi 3 B+ microcomputer (**a**); printed and mounted case for the Raspberry Pi 3 B+ computer, and cover for the Raspberry Pi Rev 2.1 camera module (**b**).

When a face is positively recognized, an output signal is sent from the GPIO port, indicating possible access for an authorized user. Based on the signals from the GPIO port, a system was built to manage access to a room through a door secured by an electromagnet controlled by the application. This required creating a simple circuit with an additional power supply and relay and an additional touch screen needed to manage the application. A positive face recognition result sends a high signal to the relay, which closes the electromagnet power supply circuit using a normally open contact, resulting in the door opening. In the case of negative recognition, the electromagnet power supply circuit remains open, keeping the door closed.

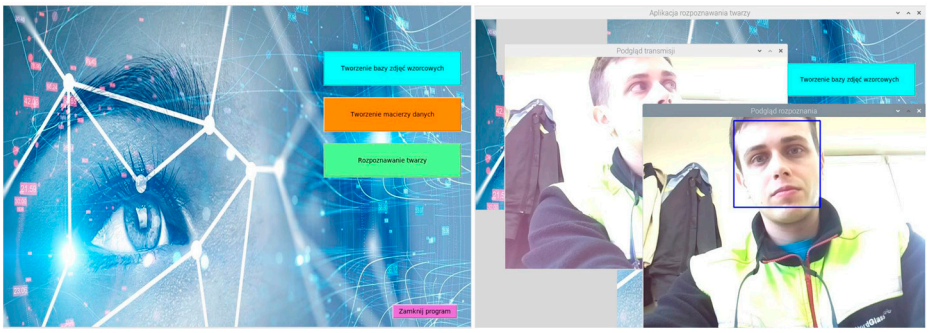
2.8. Face Recognition Application

The face recognition application (Figure 10) is divided into four separate scripts written in Python, which can be categorized as follows:

- Collecting data and sample images;
- Creating the database;
- Verifying faces;
- The graphical user interface of the application.

The application is mainly based on the "*OpenCV*" library [68], which, with its functions, enables real-time image processing obtained using the Raspberry Pi camera module. The following code allows loading of the necessary libraries used by the program.

```
from picamera.array import PiRGBArray
from picamera import PiCamera
import time
import cv2
```



(**a**) (**b**)

Figure 10. The start screen of the facial recognition application (**a**); the window with the camera transmission preview and the face detection window that collects reference photos (**b**). Blue block—Creating a database of reference photos, orange block—Creating a data matrix, green block—Face recognition, magenta block—Close program.

The "*picamera*" library is used to operate the camera module. It allows us to initialize and set the appropriate camera mode using the "*exposure_mode*" command. There are many modes to choose from, but after conducting tests, the best image quality is achieved in the "*sports*" mode. Additionally, the library allows setting the resolution, brightness, and frame rate, as shown in the code below.

```
camera = PiCamera()
camera.framerate = 24
camera.brightness = 50
camera.iso = 800
camera.exposure_mode = 'sports'
camera.exposure_compensation = 10
camera.resolution = (640, 480)
rawCapture = PiRGBArray(camera, size = (640, 480))
```

The first script for collecting sample images needs to have the image provided in grayscale. For this purpose, the command "*cv2.COLOR_BGR2GRAY*" from the "*OpenCV*" library is used.

```
skala_szarosci = cv2.cvtColor(zdjecie, cv2.COLOR_BGR2GRAY)
```

In the application, a Haar classifier is used by default, based on frontal face detection. It is loaded using the "*cv2.CascadeClassifier*" command.

```
detektor_twarzy = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

Faces of different sizes are detected using a detector, with a command running in a "*for*" loop that stops only after detecting 100 sample face images of a single user or by pressing the "*k*" key on the keyboard.

```
twarz = detektor_twarzy.detectMultiScale(skala_szarosci, 1.3, 5)
```

Next, a rectangle is created that encompasses only the face area, which is then cropped from the entire image captured by the camera module. The "*cv.rectangle*" command from the "*OpenCV*" library is used for this purpose.

```
cv2.rectangle(zdjecie, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

The rectangular area is cropped from the entire image and then saved along with the user's number and name using the "*cv.imwrite*" command in the sample image database.

```
cv2.imwrite("baza zdjęć/Użytkownik." + str(nr_uzytkownika) + '.' + str)
```

The live preview from the camera and the images of faces collected by the program are enabled by the "*cv.imshow*" command from the "*OpenCV*" library.

```
cv2.imshow('Podglad rozpoznania', zdjecie)
```

In the second script, a local binary pattern histogram is initially created, as shown below, in which a data matrix containing the mathematical description of the biometric features of the sample face images is later created (histogram training).

histogram = cv2.face.createLBPHFaceRecognizer()

histogram.train(twarze, np.array(nrID))

Similar to the histogram, an empty image file is initially created. However, unlike the data matrix, the image is overwritten with the sample image in a "*for*" loop until it is overwritten by all face images from the database. After each overwrite, a data matrix describing the given image is created. Adding a face to the empty image is performed using the "*append*" command.

probkaTwarzy.append(img_numpy[y:y + h, x:x + w])

From the sample image database, in addition to the image, the saved user data are also retrieved using the "*split*" command, which reads the appropriate information saved as the file name. In the example below, the user number, which is saved in the first position of the record row (in the first column), is read.

id = int(os.path.split(SciezkaZdjecia)[−1].split(".")[1])

After training the histogram with the data matrices of the sample images and the user data information, the histogram file is saved. It is a single text file containing all the information. It is created in the "*.yml*" format and saved using the "*save*" command.

histogram.save('baza_danych/macierz danych.yml')

The third script responsible for face recognition mainly uses the "*OpenCV*" library and its commands (similar to the script for collecting sample images). It loads the histogram created by the previous script and compares it with the data matrix of the image collected by the same Haar classifier. Depending on the recognition result, the appropriate name of the recognized person is displayed using the "*cv2.puttext*" command.

cv2.rectangle(zdjecie, (x − 22, y − 90), (x + w + 22, y − 22), (0, 255, 0), −1)

cv2.putText(zdjecie, str(Id), (x, y − 40), czcionka, 2, (255, 255, 255), 3)

The last script uses the "*Tkinter*" library, which is used for creating graphical interfaces. This allows for the creation of a separate window, specifying its size, background color, resolution, title, and many other functions, as shown in the code below. Meanwhile, the "*PIL*" library allows for loading an image from a file into the application, in this case using the "*PhotoImage*" command, which loads the image as the application's background.

program = tk.Tk(className = 'aplikacja')

program.geometry("1024x700")

program.title("Aplikacja rozpoznawania twarzy")

filename = ImageTk.PhotoImage(Image.open('/home/pi/Desktop/Aplikacja rozpoznawania twarzy'))

canvas = tk.Canvas(program, height = 1000, width = 1024)

canvas.pack()

canvas.background = filename

bg = canvas.create_image(0,0, anchor = tk.NW, image = filename)

The library also allows for creating various types of buttons, switches, text fields, and informational messages. The "*tk.Button*" command is used to create a button. The following program code shows the created buttons on the start screen.

button_1 = tk.Button(program, text = "Tworzenie bazy zdjęć wzorcowych", bg = "cyan")

button_2 = tk.Button(program, text = "Tworzenie macierzy danych", bg = "darkorange")

button_3 = tk.Button(program, text = "Rozpoznawanie twarzy", bg = "palegreen", command = proba3)

button_4 = tk.Button(program, text = "Zamknij program", bg = "orchid", command = proba3)

Using the "*os.system*" command from the "*os*" library, we can automatically run the appropriate script, for example, using a button visible on the screen. Below is a code snippet corresponding to the specification of one of the buttons.

```
def proba1():
        instrukcja1.destroy()
        button2.destroy()
        os.system('python tworzenie_bazy_danych.py')
        global button02, instrukcja02
        instrukcja02 = tk.Label(program, text = "Gotowe !")
        instrukcja02.place(x = 295, y = 205)
        button02 = tk.Button(program, text = "OK", command = proba02)
        button02.place(x = 300, y = 230)
```

The displayed messages with application instructions are implemented using the "*tk.Label*" command, which is used to create and display various types of text labels, such as a separate text window or as a label for a button.

```
def proba_0():
        global instrukcja, button1
        instrukcja = tk.Label(program, text="Zostaną wyświetlone komunikaty z prośbą
o")
        instrukcja.place(x=100, y=100)
        button1 = tk.Button(program, text = "OK", command = proba)
        button1.place(x = 350, y = 225)
```

3. Verification System

3.1. Selecting the Similarity Threshold

The similarity threshold is a value that represents the degree of biometric similarity between the tested face and the reference face. The system returns a value between 0 and 1, where a higher value indicates a greater likelihood of a correct match. The appropriate threshold must be carefully selected based on factors such as image resolution, system performance, and the specific application requirements. Any value above the set threshold is considered a match, while values below the threshold are classified as mismatches. Adjusting the threshold allows for flexibility in the system's sensitivity, balancing accuracy and performance. For applications like identifying visitors in amusement parks for souvenir photography, a lower threshold may be acceptable, as only a single correct match is required, and the risk of false positives is not critical. However, in high-security systems such as access control or banking transactions, accuracy is paramount, and a higher threshold is necessary, even if it leads to longer identification times. Selecting the appropriate similarity threshold presents a challenge because setting a value of 100% (a threshold of 1) would require the tested image to be identical to the reference image, leading to impractically long identification times and a vast number of test images. This is due to various factors affecting recognition, such as facial position, equipment quality, system performance, lighting conditions, and even minor facial changes (e.g., a day-old beard). Conversely, setting an excessively low threshold could result in the system misidentifying users by recognizing them as the reference person they resemble most biometrically, even if they are not an exact match.

To determine the optimal threshold, several tests were conducted using a group of six male individuals with varying degrees of biometric similarity.

In selecting the sample group for this study, key biometric variations known to influence facial recognition accuracy were considered. These variations included demographic

diversity, encompassing age groups such as children, young adults, middle-aged adults, and seniors. Gender representation was balanced to ensure inclusivity. Ethnic backgrounds were diverse, incorporating participants from various ethnic groups to assess algorithmic bias. Physical features, such as facial hair (e.g., beards, mustaches), and accessories like glasses, hats, or scarves, were also included. Skin tone variations and different facial expressions, including neutral, smiling, frowning, and other expressions, were tested. Additionally, unique features like scars, tattoos, or asymmetrical facial features were considered. Twelve participants took part in the study, including eight males of different ages (from 16 to 49 years old) and four females of different ages (from 19 to 51 years old). All users were entered into the database. In the following part of the article, only selected images (photos) of volunteers are presented.

These tests helped identify a suitable threshold value for achieving the desired balance between accuracy and efficiency (Figure 11).

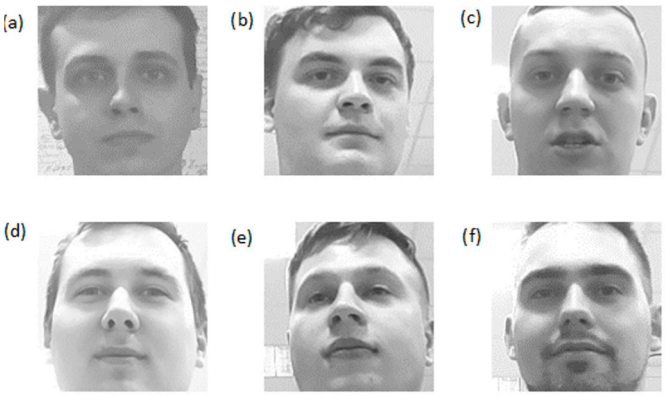


Figure 11. Test group users (photos from the application database): (**a**) User No. 1 (Kacper); (**b**) User No. 2 (Bartosz); (**c**) User No. 3 (Damian); (**d**) User No. 4 (Pawel); (**e**) User No. 5 (Patryk); (**f**) User No. 6 (Bartlomiej).

In the first test, only the first user (Kacper) was introduced into the system. Then, each subject underwent the identification process at different similarity threshold values. The results are presented in Table 1.

Table 1. The results of facial recognition with only User No. 1 (Kacper) introduced into the system and varying similarity thresholds.

Set Similarity Threshold [%]	User No. 1 (Kacper)	User No. 2 (Bartosz)	User No. 3 (Damian)	User No. 4 (Pawel)	User No. 5 (Patryk)	User No. 6 (Bartlomiej)
0	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
10	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
20	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
30	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
40	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
50	Kacper	Kacper	Kacper	Kacper	Kacper	Kacper
60	Kacper	unknown	Kacper	Kacper	Kacper	unknown
70	Kacper	unknown	Kacper	Kacper	unknown	unknown
75	Kacper	unknown	unknown	Kacper	unknown	unknown
80	Kacper	unknown	unknown	Kacper	unknown	unknown
85	Kacper	unknown	nieznana	Kacper	unknown	unknown
90	Kacper	unknown	nieznana	unknown	unknown	unknown
95	unknown	unknown	nieznana	unknown	unknown	unknown
100	unknown	unknown	unknown	unknown	unknown	unknown

The biometric similarity between User No. 1 and other users was analyzed. User No. 4 was identified as the most similar, with approximately 80% biometric similarity to User No. 1. In contrast, Users No. 2 and No. 6 were the least similar, showing about 50% similarity. When the similarity threshold was set above 90%, the system experienced difficulties recognizing the previously registered user. These challenges arose from limitations in camera resolution, system performance, variations in external conditions (e.g., identical facial expressions or positions in both images. Such factors result in variations in certain anthropometric distances, causing discrepancies between the tested image and the reference data matrix. From the initial test, it was concluded that a similarity threshold of 85–90% offers the most balanced performance. However, determining the optimal threshold cannot rely solely on identification accuracy. The recognition time is equally critical, as users should not spend excessive time adjusting their position or facial expression for a successful match. In addition to being accurate and reliable, the system must be efficient and user-friendly. Upon detecting a face in the image, the application identifies the individual by name if they are in the database or labels them as "unknown" if no match is found. To ensure usability, it was necessary to define a recognition time threshold. Identification attempts lasting more than 2 min, even with adjustments to position or expression, were considered failures. The recognition times recorded during the first test are summarized in Table 2.

Table 2. Face recognition time results (first test—Table 1) with only User No. 1 introduced into the system and varying similarity thresholds.

Set Similarity Threshold [%]	User No. 1 (Kacper) [s]	User No. 2 (Bartosz) [s]	User No. 3 (Damian) [s]	User No. 4 (Pawel) [s]	User No. 5 (Patryk) [s]	User No. 6 (Bartlomiej) [s]
0	~2	~2	~2	~2	~2	~2
10	~2	~2	~2	~2	~2	~2
20	~2	~2	~2	~2	~2	~2
30	~2	~2	~2	~2	~2	~2
40	~2	~6	~4	~2	~7	~8
50	~2	~36	~10	~3	~28	~35
60	~3	unknown	~32	~3	~114	unknown
70	~4	unknown	~74	~4	unknown	unknown
75	~4	unknown	unknown	~17	unknown	unknown
80	~9	unknown	unknown	~34	unknown	unknown
85	~16	unknown	unknown	~96	unknown	unknown
90	~29	unknown	unknown	unknown	unknown	unknown

Identification process times (Table 2) confirm that the most appropriate threshold value is in the 80–90% similarity range. Due to anthropometric distance similarities, individuals with biometric features similar to the first user (User No. 4) experience longer face recognition times despite positive recognition results at an 85% similarity threshold. We can also observe that increasing the similarity threshold causes the times to increase non-linearly, similar to exponential growth (Figure 12).

Users No. 1, 4, and 6 were entered into the database for the next test. The threshold value was varied with each identification attempt, and if the recognition time exceeded two minutes, the attempt was deemed negative. The results are recorded in Table 3.

The results of the second test (Table 3) confirm that the optimal similarity threshold is 85%. At this threshold, User No. 4 was successfully recognized even at a similarity threshold of 95%. However, User No. 6 experienced difficulty with recognition at a threshold 10% lower. A significant factor affecting these results is the variation in conditions under which reference photos were captured. For best performance, the database should be created in consistent environmental conditions and with a uniform face position (preferably

frontal). However, achieving identical conditions for all reference images is challenging. Therefore, setting the similarity threshold at 85% provides a balance between reliable recognition accuracy and relatively short identification times. For applications like access control systems, accuracy is paramount due to the potential handling of sensitive data. As such, the selected threshold must ensure both dependable operation and practical usability. In the next test, the similarity threshold was set at 85%. All test users were introduced into the system, and face recognition was conducted under varying environmental conditions and with different face positions. The results are presented in Table 4.

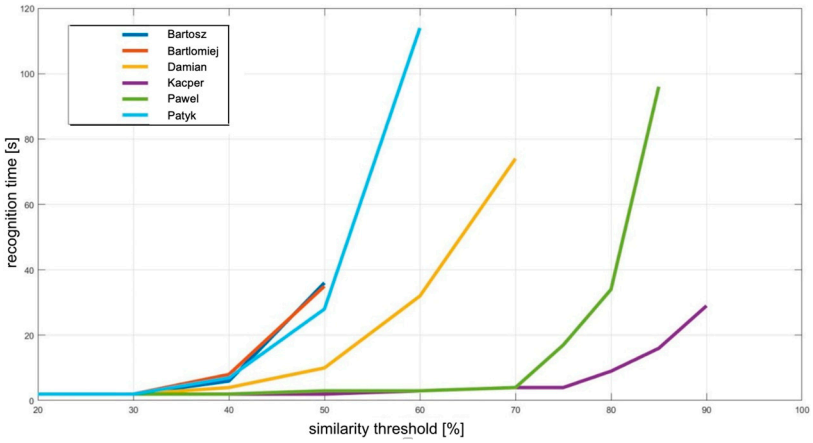


Figure 12. Graph of recognition time dependency on similarity threshold value.

Table 3. Face recognition results with only Users No. 1, 4, and 6 introduced into the system and varying similarity thresholds.

Set Similarity Threshold [%]	User No. 1 (Kacper)	User No. 2 (Bartosz)	User No. 3 (Damian)	User No. 4 (Pawel)	User No. 5 (Patryk)	User No. 6 (Bartlomiej)
0	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
10	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
20	Kacper	Bartlomiej	Pawel	Pawel	Pawel	Bartlomiej
30	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
40	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
50	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
60	Kacper	Bartlomiej	Kacper	Pawel	Pawel	Bartlomiej
70	Kacper	unknown	Kacper	Pawel	Pawel	Bartlomiej
75	Kacper	unknown	unknown	Pawel	unknown	Bartlomiej
80	Kacper	unknown	unknown	Pawel	unknown	Bartlomiej
85	Kacper	unknown	unknown	Pawel	unknown	Bartlomiej
90	Kacper	unknown	unknown	Pawel	unknown	unknown
95	unknown	unknown	unknown	Pawel	unknown	unknown
100	unknown	unknown	unknown	unknown	unknown	unknown

The results confirm that standardizing conditions for capturing test images is essential to ensure system accuracy. During the identification process, users should face the camera directly and avoid moving their heads until recognition is complete. Proper lighting conditions are crucial; face recognition should not be performed in poorly lit areas, and overly bright lighting should also be avoided as it can negatively impact results. The final test of the similarity threshold setting validated the system's performance at an 85% threshold value. For this test, each user was added to the database and subjected to three identification attempts under consistent conditions: a room with artificial lighting and users positioned frontally to the camera. The duration of the identification process

was also measured. As in previous tests, attempts lasting more than two minutes were considered unsuccessful.

Table 4. Face recognition results with all users introduced into the system and with a constant similarity threshold of 85% (conditions for capturing the tested image: A—face in a frontal position; B—face in a left-side position; C—face in a right-side position; D—face in an upward position; E—face in a downward position).

Conditions	User No. 1 (Kacper)	User No. 2 (Bartosz)	User No. 3 (Damian)	User No. 4 (Pawel)	User No. 5 (Patryk)	User No. 6 (Bartlomiej)
			In daylight			
A	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
B	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
C	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
D	Kacper	Bartosz	Damian	Pawel	unknown	Bartosz
E	Kacper	unknown	Damian	Kacper	unknown	Bartlomiej
			In artificial light			
A	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
B	Pawel	Bartosz	Damian	Pawel	unknown	Bartlomiej
C	Kacper	Bartosz	unknown	Pawel	Pawel	unknown
D	unknown	unknown	Damian	Kacper	unknown	unknown
E	Kacper	unknown	unknown	Pawel	Patryk	Bartlomiej
			In a darkened room			
A	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
B	Pawel	Bartlomiej	unknown	unknown	unknown	Bartlomiej
C	unknown	unknown	Kacper	unknown	Damian	unknown
D	unknown	unknown	unknown	unknown	unknown	unknown
E	unknown	unknown	unknown	unknown	unknown	unknown

The results are summarized in Table 5. With a fixed similarity threshold of 85% and appropriate conditions for capturing test images, the system demonstrates effective user recognition within a satisfactory time frame. Based on the conducted tests, the similarity threshold has been consistently set at 85% for anthropometric distances. However, the system's reliable operation depends on maintaining suitable conditions during image capture.

Table 5. The face recognition results obtained with all users introduced into the system and the similarity threshold set at 85%.

Attempt Number	User No. 1 (Kacper)	User No. 2 (Bartosz)	User No. 3 (Damian)	User No. 4 (Pawel)	User No. 5 (Patryk)	User No. 6 (Bartlomiej)
1	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
2	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
3	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
			Time in which the system recognized the user [s]			
1	~12	~12	~16	~9	~15	~12
2	~14	~15	~13	~13	~13	~14
3	~12	~11	~15	~12	~10	~14

3.2. Efectiveness of Recognition Depending on Facial Expressions

With a fixed similarity threshold of 85% and consistent conditions for capturing reference images (frontal face samples relative to the camera, taken under identical artificial lighting in the same room with the same background), tests were conducted to evaluate the application's effectiveness using test images with varying facial expressions (Figure 13).

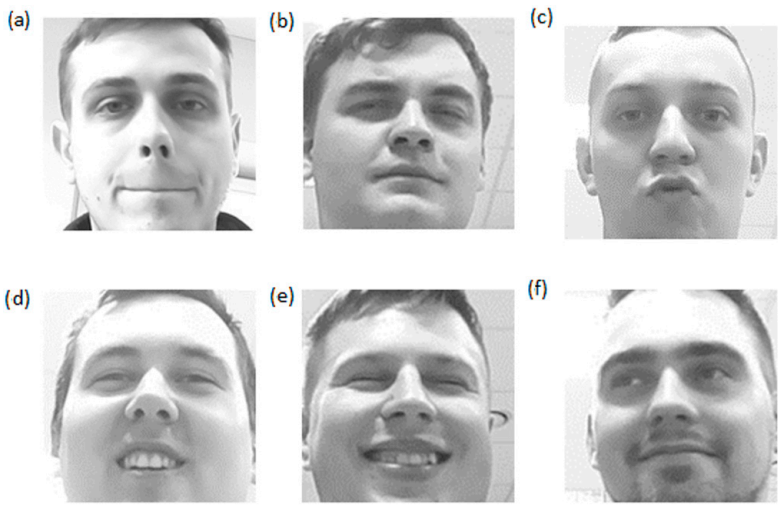


Figure 13. Sample tested images with varying facial expressions: (**a**) User No. 1 (Kacper—sadness); (**b**) User No. 2 (Bartosz—surprise); (**c**) User No. 3 (Damian—surprise); (**d**) User No. 4 (Pawel—joy); (**e**) User No. 5 (Patryk—joy); (**f**) User No. 6 (Bartlomiej—joy).

Each member of the sample group was positioned frontally to the camera. Upon initiating the process of capturing exemplary images, the user gradually moved their head to the left, then to the right, followed by upward and downward movements. Subsequently, the user altered their facial expressions to depict joy, sadness, or surprise. The system automatically concluded the image capture process upon reaching a total of one hundred photos. Users were required to complete the entire sequence of head movements and facial expression changes within this timeframe.

All reference photos were taken with neutral facial expressions, similar to those used in ID or passport photos.

For the test, the subjects attempted to replicate the same facial expression. Figure 11 shows samples of the example emotions and, therefore, the different facial representations. The test results are recorded in Table 6.

Table 6. Face recognition results in varying facial expressions.

Type of Facial Expression	User No. 1 (Kacper)	User No. 2 (Bartosz)	User No. 3 (Damian)	User No. 4 (Pawel)	User No. 5 (Patryk)	User No. 6 (Bartlomiej)
smile	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
sadness	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
surprise	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
duck face	Kacper	unknown	Damian	Pawel	unknown	Bartlomiej
clenched lips	Kacper	Bartosz	Damian	Pawel	Patryk	Bartlomiej
anger	Kacper	Bartosz	Kacper	Pawel	unknown	Bartlomiej
puffed cheeks	Pawel	Patryk	Damian	Pawel	Patryk	Bartlomiej

The application's performance with varying facial expressions can be considered satisfactory. The system consistently produced correct results for common expressions such as smiling, sadness, or surprise. However, it encountered difficulties with more exaggerated expressions, such as the "duck face". For critical applications, such as access control systems for rooms containing sensitive data, it is essential to standardize the method of capturing test images by requiring a neutral facial expression. Standardizing image capture techniques helps minimize the risk of errors. Additionally, expanding the reference image database to include frontal face positions, side angles, and various basic expressions, such as smiling or sadness, would further enhance the system's reliability.

3.3. Performance and Effectiveness Test of the Face Recognition System

The test was conducted in accordance with the established procedures for capturing reference and test images. The application's similarity threshold was set at a constant value of 85% for the data matrix. The test was performed under consistent environmental conditions, with all users registered in the database. Unlike previous tests, recognition attempts lasting longer than 30 s were classified as negative results. The outcomes are presented in Table 7.

Table 7. Face recognition system effectiveness results.

User	Recognition Result	Recognition Time [s]
User No. 1 (Kacper—25 years old)	Kacper	~11
User No. 2 (Maciej—29 years old)	Maciej	~10
User No. 3 (Wojciech—43 years old)	Wojciech	~9
User No. 4 (Janusz—49 years old)	Janusz	~12
User No. 5 (Marcin—35 years old)	Marcin	~11
User No. 6 (Piotr—39 years old)	Piotr	~9
User No. 7 (Dawid—27 years old)	Dawid	~12
User No. 8 (Filip—16 years old)	Filip	~8
User No. 9 (Olimpia—19 years old)	Olimpia	~11
User No. 10 (Dorota—51 years old)	Dorota	~10
User No. 11 (Beata—46 years old)	Beata	~10
User No. 12 (Karolina—22 years old)	Karolina	~14

The results confirm the system's accuracy regardless of variations in user age and gender. The average recognition time (Figure 14) is 10.583 s, which is considered satisfactory. Following the procedures outlined in the previous subsection, the system is suitable for critical applications, such as controlling access to rooms containing sensitive data. However, it is recommended to implement a maximum identification time of 30 s in the program as an additional safeguard against incorrect recognition results, complementing the similarity threshold. While this limit enhances reliability, it could lead to user frustration in cases of identification failure within the allotted time, even if the user is registered in the system. Such failures could stem from non-compliance with image-capture procedures or changes in daily appearance, such as different makeup or facial hair growth.

Maintaining consistent environmental conditions in an industrial face identification process is highly challenging. Additionally, biometric facial similarities among individuals can complicate the development of an infallible face recognition system. Future research could explore the impact of facial aging on the identification process, helping to determine when users should update their reference images in the database to minimize the risk of non-recognition despite prior verification. Another important consideration is the system's performance with a significantly larger database, such as one containing over a thousand users. This scenario would likely increase recognition times and require greater computational power.

Despite these challenges, the system has demonstrated effectiveness and efficiency for a small group of users (tested with 12 individuals) when proper verification procedures are followed. A final test assessed the program's resilience against spoofing by substituting user photos for live camera input. This test evaluated the system's ability to detect and prevent fraudulent attempts. The test was conducted with the same group of 12 users, using ID card photos to simulate frontal face positioning relative to the camera lens. The results are summarized in Table 8.

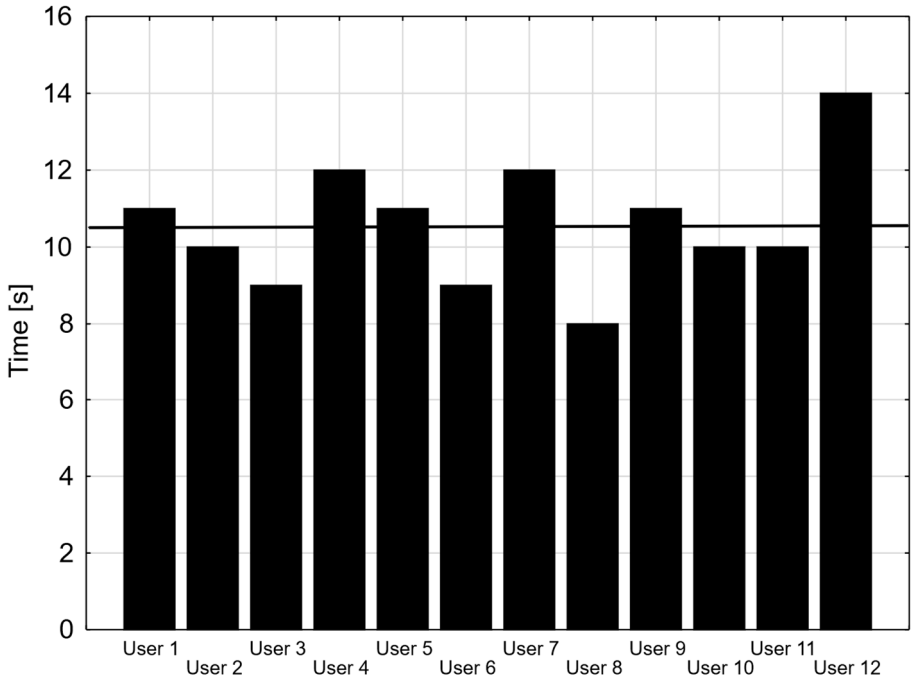


Figure 14. Sample tested images with varying facial expressions. Horizontal black line—the average recognition time 10.583 s.

Table 8. User identification results are based on their photo.

User	Recognition Result	Recognition Time [s]
User No. 1 (Kacper—25 years old)	Kacper	~15
User No. 2 (Maciej—29 years old)	unknown	-
User No. 3 (Wojciech—43 years old)	Wojciech	~11
User No. 4 (Janusz—49 years old)	Janusz	~24
User No. 5 (Marcin—35 years old)	Marcin	~11
User No. 6 (Piotr—39 years old)	unknown	-
User No. 7 (Dawid—27 years old)	Dawid	~20
User No. 8 (Filip—16 years old)	Filip	~16
User No. 9 (Olimpia—19 years old)	Olimpia	~9
User No. 10 (Dorota—51 years old)	Dorota	~19
User No. 11 (Beata—46 years old)	unknown	-
User No. 12 (Karolina—22 years old)	Karolina	~10

The results confirm the ease of tricking the system, which eliminates it as a security system for so-called "hard" processes, where the possibility of error is unacceptable. Recognition times for printed photos were comparable to those for live user identification. However, the system could be enhanced by integrating a thermal imaging camera to verify whether the image being analyzed originates from a living individual generating heat or from a photograph.

4. Conclusions

The main goal of this project was to design a face detection and identification system capable of autonomously creating and utilizing a reference image database. This objective was successfully achieved, as the system demonstrated solid performance throughout testing. Built on a Raspberry Pi microcomputer, the application operates efficiently, achieving an average recognition time of 10.5 s. When standardized procedures for capturing reference and test images are followed under controlled environmental conditions, the system delivers near-flawless accuracy, as confirmed by the test results. Although the

system performs well for small user groups and controlled scenarios, there are several areas where further research is needed to improve its robustness and scalability:

- Database Size and Biometric Variations: To assess scalability and reliability, the system needs to be tested with larger databases, exceeding a thousand users, including individuals with significant biometric similarities.
- Hardware Impact: The current setup uses a standard Raspberry Pi-compatible camera module. Initial tests suggest that higher-resolution cameras could improve accuracy and facial feature detection, particularly when the reference images are of higher quality than the test images. Exploring the impact of hardware upgrades on system performance, especially under challenging conditions like poor lighting or occlusions, would be a valuable next step.
- Algorithm and Library Performance: The system employs the Haar cascade classifier for face detection and data matrix generation, achieving high accuracy with frontal faces. However, its performance decreases with extreme facial angles. Leveraging additional tools from the OpenCV library, such as deep learning-based models, could mitigate these limitations and broaden the system's capabilities.
- Security Enhancements: While the system works well for access control in environments dealing with sensitive data, it is vulnerable to spoofing via printed photographs. Adding features like thermal imaging to differentiate living faces from static images or incorporating additional biometric verifications (e.g., fingerprint or retina scans) would enhance security.

Beyond access control, the system's flexibility enables a range of potential applications:

- Retail and Surveillance: tracking customer movements or behavior in stores.
- Data Collection: gathering pedestrian statistics or analyzing consumer habits.
- Media Analysis: measuring screen time for actors in media productions.

Although these extensions were beyond the original scope of the project, they highlight the system's adaptability for future development [69,70]. Implementing such features would require additional modules, like object tracking, and further refinement of the algorithms. This system has proven to be a reliable facial recognition tool under specific conditions [71]. While it may not be suited for high-security environments requiring zero errors, it provides a strong foundation for further innovation in facial recognition technology [72]. With the proposed improvements and adaptations, the system has the potential to be expanded into a variety of use cases, ranging from access control to data analytics and surveillance.

Author Contributions: Conceptualization, S.P., A.B. and K.G.; methodology, S.P., T.K. and A.B.; software, K.G., A.B., S.P. and I.M.; validation, S.G. and I.M.; formal analysis, S.G., I.M. and T.K.; investigation, S.P. and A.B.; resources, K.G., S.P. and A.B.; data curation, K.G., A.B. and S.P.; writing—original draft preparation, K.G., S.P., I.M., A.B. and S.G.; writing—review and editing, A.B., S.P., S.G. and I.M.; visualization, K.G., I.M. and T.K.; supervision, S.P., A.B. and S.G.; project administration, S.G. and I.M.; funding acquisition, T.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: The technical development and testing of a recognition algorithm, it does not fall under the category of human research as defined by the Declaration of Helsinki. Therefore, institutional review board (IRB) approval was not required.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: Available by e-mail: sebastian.pecolt@tu.koszalin.pl.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhao, W.; Chellappa, R.; Phillips, P.J.; Rosenfeld, A. Face Recognition: A Literature Survey. *ACM Comput. Surv.* **2003**, *35*, 399–458. [CrossRef]
2. Chethan, K.; Sachin Kumar, A. A Literature Survey on Online Voting System Using Face Recognition. *Int. J. Adv. Res. Sci. Commun. Technol.* **2024**, *4*, 1–10. [CrossRef]
3. Turk, M.; Pentland, A. Eigenfaces for Recognition. *J. Cogn. Neurosci.* **1991**, *3*, 71–86. [CrossRef] [PubMed]
4. Ahonen, T.; Hadid, A.; Pietikäinen, M. Face Recognition with Local Binary Patterns. In Proceedings of the ECCV 2004: Computer Vision—ECCV 2004, Prague, Czech Republic, 11–14 May 2004; Pajdla, T., Matas, J., Eds.; Springer: Berlin/Heidelberg, Germany, 2004; pp. 469–481. [CrossRef]
5. Jain, A.K.; Ross, A.; Prabhakar, S. An Introduction to Biometric Recognition. *IEEE Trans. Circuits Syst. Video Technol.* **2004**, *14*, 4–20. [CrossRef]
6. Introna, L.D.; Nissenbaum, H. *Facial Recognition Technology: A Survey of Policy and Implementation Issues*; Center for Catastrophe Preparedness and Response, New York University: New York, NY, USA, 2009; Volume 74, pp. 1–36.
7. Kostka, G.; Steinacker, L.; Meckel, M. Between Security and Convenience: Facial Recognition Technology in the Eyes of Citizens in China, Germany, the United Kingdom, and the United States. *Public Underst. Sci.* **2021**, *30*, 644–659. [CrossRef]
8. Bennett, C.J. Surveillance Society: Monitoring Everyday Life, by Lyon, D., Buckingham and Philadelphia: Open University Press, 2001. xii+ 189 pp. $27.95 (paper). ISBN 0-33520546-1. *The Inf. Soc.* **2003**, *19*, 335–336. [CrossRef]
9. Jain, A.K.; Bolle, R.; Pankanti, S. *Biometrics: Personal Identification in Networked Society*; Springer: Boston, MA, USA, 2006.
10. Pushparani, M.; Indumathi, T. Human Authentication by Matching 3D Skull with Face Image Using SCCA. *Int. J. Appl. Eng. Res.* **2015**, *10*, 34247–34254.
11. Wang, S.; Liu, J. Biometrics on Mobile Phone. In *Recent Application in Biometrics*; IntechOpen: London, UK, 2011. [CrossRef]
12. Manonmani, S.P.; Abirami, G.; Sri, V.N. Spoof Prevention for E-Banking Using Live Face Recognition. *Int. Res. J. Mod. Eng. Technol. Sci.* **2023**, *5*, 4600–4603. [CrossRef]
13. Nosrati, L.; Bidgoli, A.M.; Javadi, H.H.S. Identifying People's Faces in Smart Banking Systems Using Artificial Neural Networks. *Int. J. Comput. Intell. Syst.* **2024**, *17*, 9. [CrossRef]
14. Mohanraj, K.C.; Ramya, S.; Sandhiya, R. Face Recognition-Based Banking System Using Machine Learning. *Int. J. Health Sci.* **2022**, *6*, 19724–19730. [CrossRef]
15. Brunelli, R.; Poggio, T. Face Recognition: Features Versus Templates. *IEEE Trans. Pattern Anal. Mach. Intell.* **1993**, *15*, 1042–1052. [CrossRef]
16. Bouhou, L.; el Ayachi, R.; Baslam, M.; Oukessou, M. Face Recognition in a Mixed Document Based on the Geometric Method. *Int. J. Adv. Sci. Technol.* **2018**, *116*, 115–124. [CrossRef]
17. Hidayat, R.; Wibowo, M.O.B.; Satria, B.Y.; Winursito, A. Implementation of Face Recognition Using Geometric Features Extraction. *J. Ilmiah Kursor* **2022**, *11*, 2. [CrossRef]
18. Lee, H.; Park, S.H.; Yoo, J.H.; Jung, S.H.; Huh, J.H. Face Recognition at a Distance for a Stand-Alone Access Control System. *Sensors* **2020**, *20*, 785. [CrossRef]
19. Sajja, T.K.; Kalluri, H.K. Face Recognition Using Local Binary Pattern and Gabor-Kernel Fisher Analysis. *Int. J. Adv. Intell. Paradigms* **2023**, *26*, 28–42. [CrossRef]
20. Allagwail, S.; Gedik, O.S.; Rahebi, J. Face Recognition with Symmetrical Face Training Samples Based on Local Binary Patterns and the Gabor Filter. *Symmetry* **2019**, *11*, 157. [CrossRef]
21. Verma, S.B.; Tiwari, N. Local Binary Patterns Histograms (LBPH) Based Face Recognition. *Int. J. Eng. Adv. Technol.* **2019**, *9*, 1088–1091. [CrossRef]
22. Vu, H.N.; Nguyen, M.H.; Pham, C. Masked Face Recognition with Convolutional Neural Networks and Local Binary Patterns. *Appl. Intell.* **2022**, *52*, 2394–2405. [CrossRef]
23. Tang, H.; Yin, B.; Sun, Y.; Hu, Y. 3D Face Recognition Using Local Binary Patterns. *Signal Process.* **2013**, *93*, 2087–2096. [CrossRef]
24. Taigman, Y.; Yang, M.; Ranzato, M.; Wolf, L. DeepFace: Closing the Gap to Human-Level Performance in Face Verification. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 23–28 June 2014. [CrossRef]
25. Schroff, F.; Kalenichenko, D.; Philbin, J. FaceNet: A Unified Embedding for Face Recognition and Clustering. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 7–12 June 2015.
26. Chen, Y.; Zhang, M. Research on Face Emotion Recognition Algorithm Based on Deep Learning Neural Network. *Appl. Math. Nonlinear Sci.* **2024**, *9*, 1–16. [CrossRef]
27. Medjdoubi, A.; Meddeber, M.; Yahyaoui, K. Smart City Surveillance: Edge Technology Face Recognition Robot Deep Learning Based. *Int. J. Eng. Trans. A Basics* **2024**, *37*, 3. [CrossRef]

28. Guo, G.; Zhang, N. A Survey on Deep Learning Based Face Recognition. *Comput. Vis. Image Underst.* **2019**, *189*, 102805. [CrossRef]
29. Kasim, N.A.B.M.; Rahman, N.H.B.A.; Ibrahim, Z.; Mangshor, N.N.A. Celebrity Face Recognition Using Deep Learning. *Indones. J. Electr. Eng. Comput. Sci.* **2018**, *12*, 476–481. [CrossRef]
30. Alzu'bi, A.; Albalas, F.; Al-Hadhrami, T.; Younis, L.B.; Bashayreh, A. Masked Face Recognition Using Deep Learning: A Review. *Electronics* **2021**, *10*, 2666. [CrossRef]
31. Braga, F.M.F.; Cardoso, P.V. O'Neil, Cathy. Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy. Crown: New York, 2016. *Mural Int.* **2017**, *7*, 94–97. [CrossRef]
32. Li, W.; Hua, M.; Sun, Y.; Li, H.; Lin, Y. Face, Facial Recognition Technology and Personal Privacy. *Acta Bioethica* **2023**, *29*, 2. [CrossRef]
33. Tomaz Ketley, I. Case Study: Code of Ethics for Facial Recognition Technology. In *Proceedings of the Wellington Faculty of Engineering Ethics and Sustainability Symposium*; Te Herenga Waka-Victoria University of Wellington: Wellington, New Zealand, 2022. [CrossRef]
34. Chan, J. Facial Recognition Technology and Ethical Issues. In *Proceedings of the Wellington Faculty of Engineering Ethics and Sustainability Symposium*; Te Herenga Waka-Victoria University of Wellington: Wellington, New Zealand, 2022. [CrossRef]
35. North-Samardzic, A. Biometric Technology and Ethics: Beyond Security Applications. *J. Bus. Ethics* **2020**, *167*, 3. [CrossRef]
36. Buolamwini, J.; Gebru, T. Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification. In Proceedings of the 1st Conference on Fairness, Accountability and Transparency, New York, NY, USA, 23–24 February 2018.
37. Raji, I.D.; Buolamwini, J. Actionable auditing: Investigating the impact of publicly naming biased performance results of commercial AI products. In Proceedings of the AIES 2019—Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society, Honolulu, HI, USA, 27–28 January 2019. [CrossRef]
38. Liang, H.; Perona, P.; Balakrishnan, G. Benchmarking Algorithmic Bias in Face Recognition: An Experimental Approach Using Synthetic Faces and Human Evaluation. In Proceedings of the 2023 IEEE/CVF International Conference on Computer Vision (ICCV), Paris, France, 1–6 October 2023. [CrossRef]
39. Schuetz, P.N.K. Fly in the Face of Bias: Algorithmic Bias in Law Enforcement's Facial Recognition Technology and the Need for an Adaptive Legal Framework. *Minn. J. Law Inequal.* **2021**, *39*, 221. [CrossRef]
40. Serna, I.; Morales, A.; Fierrez, J.; Obradovich, N. Sensitive loss: Improving accuracy and fairness of face representations with discrimination-aware deep learning. *Artif. Intell.* **2022**, *305*, 103682. [CrossRef]
41. Klontz, J.C.; Jain, A.K. A case study of automated face recognition: The Boston marathon bombings suspects. *Computer* **2013**, *46*, 91–94. [CrossRef]
42. Naser, O.A.; Ahmad, S.M.S.; Samsudin, K.; Hanafi, M.; Shafie, S.M.B.; Zamri, N.Z. Facial recognition for partially occluded faces. *Indones. J. Electr. Eng. Comput. Sci.* **2023**, *30*, 1846–1855. [CrossRef]
43. da Silva, J.R.; de Almeida, G.M.; Cuadros, M.A.d.S.L.; Campos, H.L.M.; Nunes, R.B.; Simão, J.; Muniz, P.R. Recognition of Human Face Regions under Adverse Conditions—Face Masks and Glasses—In Thermographic Sanitary Barriers through Learning Transfer from an Object Detector. *Machines* **2022**, *10*, 43. [CrossRef]
44. Lin, S.D.; Chen, L.; Chen, W. Thermal face recognition under different conditions. *BMC Bioinform.* **2021**, *22*, 313. [CrossRef] [PubMed]
45. Goswami, G.; Vatsa, M.; Singh, R. RGB-D face recognition with texture and attribute features. *IEEE Trans. Inf. Forensics Secur.* **2014**, *9*, 1629–1640. [CrossRef]
46. Jain, A.K.; Ross, A.; Nandakumar, K. *Introduction to Biometrics*; Springer: Berlin/Heidelberg, Germany, 2011; ISBN 978-0-387-77325-4.
47. Ekman, P.; Friesen, W. Facial action coding system: A technique for the measurement of facial movement. In *Differences Among Unpleasant Feelings, Motivation and Emotion*; Palo Alto: Santa Clara, CA, USA, 1978.
48. Alisawi, M.; Yalçin, N. Real-Time Emotion Recognition Using Deep Learning Methods: Systematic Review. *Intell. Methods Eng. Sci.* **2023**, *2*, 5–21. [CrossRef]
49. Chavali, T.S.; Kandavalli, T.C.; Sugash, T.M.; Subramani, R. Smart Facial Emotion Recognition with Gender and Age Factor Estimation. *Procedia Comput. Sci.* **2022**, *218*, 113–123. [CrossRef]
50. Dores, A.R.; Barbosa, F.; Queirós, C.; Carvalho, I.P.; Griffiths, M.D. Recognizing emotions through facial expressions: A large-scale experimental study. *Int. J. Environ. Res. Public Health* **2020**, *17*, 7420. [CrossRef]
51. Dhana Ranjini, M.M.; Jeyaraj, M.P.; Kumar, M.S.; Prasath, T.A.; Prabhakar, G. Haar Cascade Classifier-based Real-Time Face Recognition and Face Detection. In Proceedings of the 4th International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 6–8 July 2023; pp. 990–995. [CrossRef]
52. Zhang, C.; Liu, G.; Zhu, X.; Cai, H. Face Detection Algorithm Based on Improved AdaBoost and New Haar Features. In Proceedings of the 12th International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI), Suzhou, China, 19–21 October 2019; pp. 1–5. [CrossRef]

53. Knoche, M.; Hörmann, S.; Rigoll, G. Cross-quality LFW: A database for analyzing cross-resolution image face recognition in unconstrained environments. In Proceedings of the 16th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2021, Jodhpur, India, 15–18 December 2021; pp. 1–5.
54. Richoz, A.R.; Stacchi, L.; Schaller, P.; Lao, J.; Papinutto, M.; Ticcinelli, V.; Caldara, R. Recognizing facial expressions of emotion amid noise: A dynamic advantage. *J. Vis.* **2024**, *24*, 7. [CrossRef]
55. Kanade, T.; Cohn, J.F.; Tian, Y. Comprehensive database for facial expression analysis. In Proceedings of the 4th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2000, Grenoble, France, 28–30 March 2000. [CrossRef]
56. Tome, P.; Vera-Rodriguez, R.; Fierrez, J.; Ortega-Garcia, J. Facial Soft Biometric Features for Forensic Face Recognition. *Forensic Sci. Int.* **2015**, *257*, 271–284. [CrossRef]
57. Arcoverde, E.; Duarte, R.; Barreto, R.; Magalhaes, J.; Bastos, C.; Ing Ren, T.; Cavalcanti, G. Enhanced Real-time Head Pose Estimation System for Mobile Devices. *Integr. Comput. Aided Eng.* **2014**, *21*, 281–293. [CrossRef]
58. Nalepa, J.; Szymanek, J.; Kawulok, M. Real-time People Counting from Depth Images. In Proceedings of the Beyond Databases, Architectures and Structures: 11th International Conference, BDAS 2015, Ustroń, Poland, 26–29 May 2015; pp. 387–397. [CrossRef]
59. Girshick, R.; Iandola, F.; Darrell, T.; Malik, J. Deformable Part Models Are Convolutional Neural Networks. *arXiv* **2014**. [CrossRef]
60. Paul, T.; Shammi, U.A.; Kobashi, S. A Study on Face Detection Using Viola-Jones Algorithm in Various Backgrounds, Angles and Distances. *Int. J. Biomed. Soft Comput. Hum. Sci. Off. J. Biomed. Fuzzy Syst. Assoc.* **2018**, *23*, 27–36.
61. Taskar, B.; Guestrin, C.; Koller, D. Max-margin Markov Networks. *Adv. Neural Inf. Process. Syst.* **2003**, *16*.
62. Gu, H.; Su, G.; Du, C. Feature Points Extraction from Faces. *Image Vis. Comput.* **2003**, *26*, 154–158.
63. Jaadi, Z. Principal Component Analysis (PCA) Explained. Built In. 2022. Available online: https://builtin.com/data-science/step-step-explanation-principal-component-analysis (accessed on 21 November 2024).
64. Fei, H.; Fan, Z.; Wang, C.; Zhang, N.; Wang, T.; Chen, R.; Bai, T. Cotton Classification Method at the County Scale Based on Multi-Features and Random Forest Feature Selection Algorithm and Classifier. *Remote Sens.* **2022**, *14*, 829. [CrossRef]
65. Huang, C.L.; Chen, C.W. Human Facial Feature Extraction for Face Interpretation and Recognition. *Pattern Recognit.* **1992**, *25*, 1435–1444. [CrossRef]
66. McCormick, M.L. AdaBoost Tutorial. 2013. Available online: http://mccormickml.com/2013/12/13/adaboost-tutorial/ (accessed on 22 November 2024).
67. Murtaza, M.; Sharif, M.; Raza, M.; Shah, J.H. Analysis of Face Recognition under Varying Facial Expression: A Survey. *Int. Arab. J. Inf. Technol.* **2013**, *10*, 378–388.
68. OpenCV. Cascade Classifier Tutorial. Available online: https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html (accessed on 22 November 2024).
69. Ashtagi, R.; Malse, N.; Mohite, S.; Nipanikar, S.; Lanjewar, R. Face Recognition-Based Attendance System. In *Advances in Information Communication Technology and Computing*; Goar, V., Kuri, M., Kumar, R., Senjyu, T., Eds.; Springer: Singapore, 2024; Volume 1074, pp. 993–998. [CrossRef]
70. Howard, J.J.; Sirotin, Y.B.; Vemury, A.R. The Effect of Broad and Specific Demographic Homogeneity on the Imposter Distributions and False Match Rates in Face Recognition Algorithm Performance. In Proceedings of the 10th International Conference on Biometrics Theory, Applications and Systems (BTAS 2019), Tampa, FL, USA, 23–26 September 2019. [CrossRef]
71. Glowinski, S.; Pecolt, S.; Błażejewski, A.; Sobieraj, M. Design of a Low-Cost Measurement Module for the Acquisition of Analogue Voltage Signals. *Electronics* **2023**, *12*, 610. [CrossRef]
72. Dang, T.M.; Nguyen, T.D.; Hoang, T.; Kim, H.; Teoh, A.B.J.; Choi, D. AVET: A Novel Transform Function to Improve Cancellable Biometrics Security. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 758–772. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.